

Tema 10.- Computación evolutiva

Orígenes y bases biológicas

La Computación Evolutiva (CE) nace de intentar replicar la forma en que los seres vivos (los sistemas más complejos conocidos) resuelven su principal problema: la supervivencia.

Las 4 teorías evolutivas base

Para entender el algoritmo, primero hay que saber qué conceptos biológicos imita:

- Lamarck: Propuso la herencia de los caracteres adquiridos.
- Darwin: Introdujo la selección natural (sobreviven los más aptos).
- Mendel: Explicó el mecanismo de la herencia genética.
- De Vries: Observó la mutación como cambios abruptos.
- Neo-darwinismo: es simplemente la suma de todas las teorías anteriores. Es el paradigma exacto en el que se basan los Algoritmos Genéticos.
- Teoría naturalista de evolución molecular: deriva genética

Historia en la informática

John Holland (1975): Es el verdadero creador de la teoría de los AG. Su objetivo original era lograr que los programas aprendieran por sí mismos adaptándose al medio.

David Goldberg (1989): Fue alumno de Holland y el pionero en aplicar estos algoritmos para resolver problemas industriales.

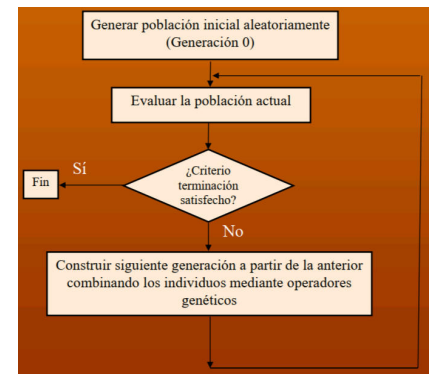
Clasificación y naturaleza de los AG

En la taxonomía de los métodos de búsqueda computacionales, los Algoritmos Genéticos se clasifican como un método de búsqueda aleatorio. No son exhaustivos (no prueban todo) ni están basados en el cálculo directo de derivadas.

Son ideales para resolver problemas NP-duros (sin solución algorítmica directa polinomial) con espacios de búsqueda enormes y superficies complejas llenas de mínimos locales.

Los 5 componentes de un AG

1. Representación genética (Codificación): Transformar la solución del problema en una cadena de bits o caracteres (el cromosoma).
2. Población Inicial: Se genera de forma aleatoria para asegurar variedad.
3. Función de Evaluación (Fitness): Es el "juez". Evalúa qué tan buena es una solución. "Premia" a las buenas permitiendo que se propaguen y "castiga" a las malas.
4. Operadores Genéticos: Mecanismos para cambiar la composición de los descendientes (Cruce y Mutación).
5. Parámetros de control: Tamaño de la población, probabilidad de cruce (P_c) y probabilidad de mutación (P_m).

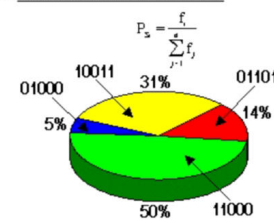


Criterio de selección

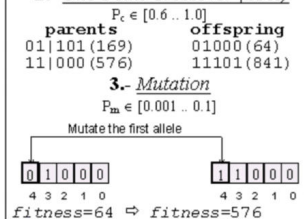
Decide qué individuos de la población actual se cruzarán para la siguiente generación.

- Ruleta: La probabilidad de ser elegido es proporcional a su fitness ($F_s = f_i / \sum_{j=1} f_j$).
 - Ejerce mucha presión sobre el mejor individuo, favoreciendo la explotación (refinar buenas soluciones en un área local).
- Torneo: Selecciona un número al azar de individuos y coge al más adaptado de ese grupo.
 - Reparte la búsqueda de forma más equitativa, evitando que un super-individuo acapare todo. Favorece la exploración (buscar en áreas nuevas del espacio de estados).

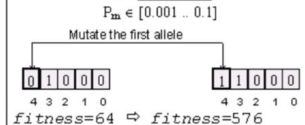
1.- Roulette Wheel Selection



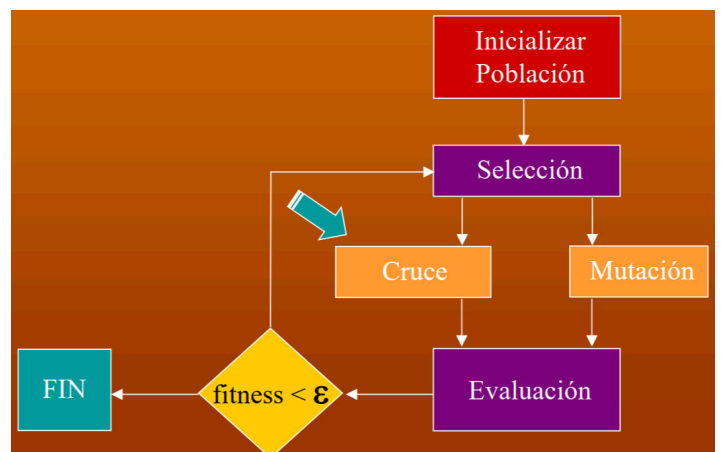
2.- One-Point Crossover (SPX)



3.- Mutation



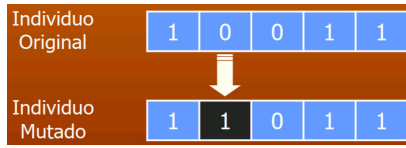
Tipo de operador	Algoritmo	Comentario
Selección de individuos	Torneo	Reparte la búsqueda más en el espacio de estados (exploración)
	Sobranje estocástico	Realizan la búsqueda de forma intermedia entre ambos
	Universal estocástica	
	Muestreo determinístico	
	Ruleta	Pone más presión de búsqueda sobre el mejor individuo (explotación)



Cruce

Mezcla el material genético de dos padres. Su objetivo es reunir en un individuo lo mejor de sus progenitores.

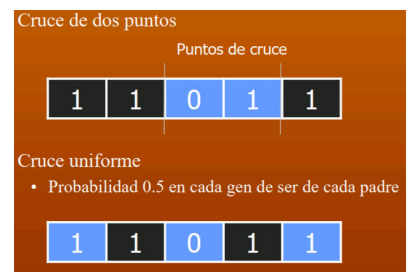
- Cruce de un punto / dos puntos: Se corta la cadena y se intercambian los fragmentos.
- Cruce uniforme: En cada gen (bit), hay un 50% de probabilidad de heredar de un padre o del otro.
- Efecto: El cruce busca combinaciones óptimas (Explotación).



Mutación

Consiste en invertir o cambiar un bit al azar basado en una probabilidad baja.

- ¿Por qué es crucial? Contribuye a la diversidad genética y es el único operador capaz de sacar al algoritmo de máximos/mínimos locales.
- Efecto: La mutación busca variabilidad (Exploración).



Gestión de la población y criterios

Sustitución (Cómo avanzamos de generación):

- Generacional: Se genera una población totalmente nueva que sustituye a la anterior.
- Steady-state: Solo se sustituye a los peores individuos, provocando homogeneización rápida.
- Elitismo: Consiste en asegurar que los N mejores individuos pasen a la siguiente generación intactos, para no perder información valiosa por culpa del cruce o la mutación.

Criterios de paro

El algoritmo termina cuando se alcanza un porcentaje alto de convergencia (toda la población es muy similar). Si ocurre esto y no hemos llegado a la solución deseada, se suele inyectar "diversidad genética" (mutar drásticamente) o se reemplaza la población.

Parámetros Típicos:

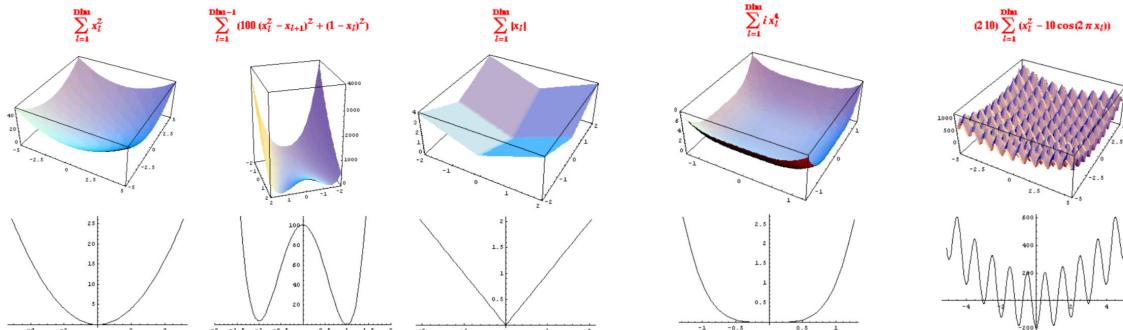
- Cruce (P_c): Suele ser muy alto (alrededor del 90%).
- Mutación (P_m): Suele ser muy bajo (5-10%) para no convertir el algoritmo en una simple búsqueda aleatoria descontrolada.

Casos de estudio de los AG

Los Algoritmos Genéticos (AG) no solo sirven para buscar el mínimo de una parábola. Se aplican en optimización de funciones, investigación operativa (ej. logística de transportes), sistemas clasificadores y búsqueda de rutas.

Optimización de funciones: el entorno de De Jong

¿Cómo sabemos si un Algoritmo Genético es bueno o malo? En 1975, Kenneth De Jong (en su tesis doctoral) construyó un entorno de prueba estandarizado por 5 problemas de minimización de funciones.

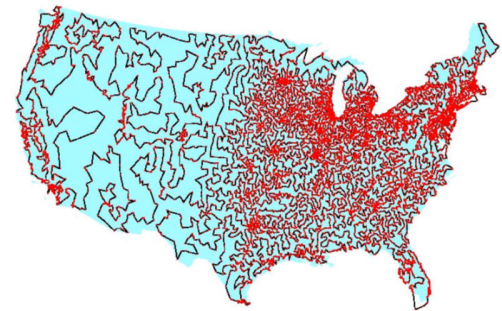


El problema del viajante (TSP)

El problema: Encontrar la ruta más corta (o de menor tiempo) para visitar un número N de ciudades volviendo al punto de origen sin repetir ninguna ciudad. Es un problema NP-duro clásico.

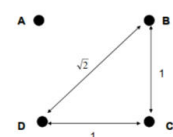
Hitos históricos

- 1962: Concurso de "Car 54" para 33 ciudades de EEUU.
- 1988: Holland y Groetschel encuentran la ruta óptima para 666 lugares del mundo usando técnicas evolutivas.
- 1998: Se logra la ruta óptima para las 13509 ciudades de EEUU de más de 500 habitantes.



Mecánica del AG

- Codificación: Se usan permutaciones de ciudades (Ej: 3-5-1-4-2).
- Operadores Especiales:
 - Cruce: No sirve el clásico. Se debe tomar una parte de un padre (ej. las primeras 3 ciudades) y rellenar el resto manteniendo el orden relativo de las ciudades que faltan basándose en el segundo padre.
 - Mutación: Simplemente se cambia el orden (se intercambian dos ciudades de posición).
- El problema del Estancamiento: Si la población converge prematuramente (todos los individuos proponen la misma ruta), la solución es partir de una población inicial muy grande e inyectar mutación.



DISTANCIAS	
AB	1
BC	1
CD	1
DA	1
AC	$\sqrt{2}$
BD	$\sqrt{2}$

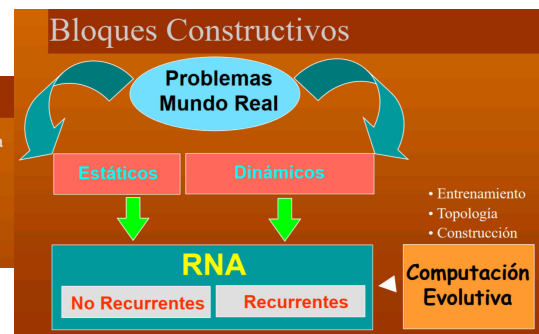
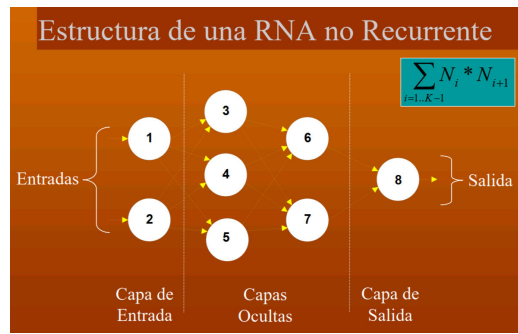
Nº de caminos recorridos igual a 4

$$Fitness(w)_1 = 1 - \frac{C - \sum_{i=0}^{L-1} b_i}{C}$$

La distancia recorrida debe ser mínima

$$Fitness(w)_2 = \frac{\sum_{i=0}^{L-1} b_i T_i}{\sum_{i=0}^{L-1} b_i}$$

$$Fitness(w) = Fitness(w)_1 - 0.1 * Fitness(w)_2$$



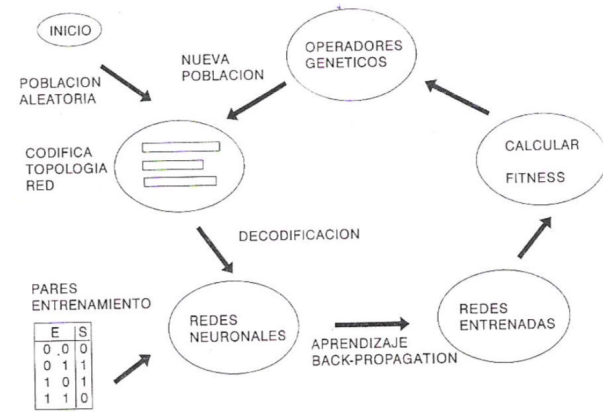
Entrenamiento: Optimización de pesos

- En lugar de usar el algoritmo matemático de Backpropagation (que usa derivadas y cálculo de errores), el AG asume que el individuo (el cromosoma) es un array que contiene todos los pesos de la red (V13, V14...).
- Ventaja clave: Funciona para redes no recurrentes y, lo más importante, para redes recurrentes (aquellas donde la información forma bucles, que son muy difíciles de entrenar matemáticamente).
- Fitness: Minimizar el Error Cuadrático Medio (ECM).

Búsqueda de topología

Busca la estructura óptima (cuántas neuronas, cuántas capas, quién se conecta con quién).

- Codificación Directa: El individuo es una matriz bidimensional (traducida a un array lineal) de 0s y 1s. Si hay un 1, hay conexión entre esas dos neuronas.
- La trampa del diseño: Si dejas al AG evolucionar libremente, creará redes gigantescas conectadas con todo porque matemáticamente tendrán menos error. Para evitar esto, la función fitness introduce una penalización: $\text{Fitness} = \text{ECM} + \text{número de conexiones no 0}$. Así obligamos al algoritmo a buscar redes precisas pero sencillas.



Problema de satisfacción lógica: FNC

Este es un problema lógico puro (Formas Normales Conjuntivas). Consiste en encontrar los valores de verdad (True/False) de un conjunto de variables para que toda una ecuación lógica gigantesca devuelva True.

- Codificación: Un individuo es una cadena de bits (ej: 101010) donde cada bit representa si la variable A, B, C... es Verdadera (1) o Falsa (0).
- Función Fitness: Se evalúa la ecuación lógica por partes (cláusulas). El fitness es simplemente un contador: el número de cláusulas que el individuo logra satisfacer (hacer verdaderas). El algoritmo terminará cuando el fitness alcance el máximo posible (todas las cláusulas son True).

Conclusiones

Diferencias con métodos tradicionales

- No trabajan con los parámetros del problema, sino con una codificación de los mismos (ej: código binario).
- No inician la búsqueda desde un punto, sino desde una población (múltiples puntos simultáneamente).
- No usan derivadas matemáticas ni gradientes, solo la función fitness (evaluación).
- Sus reglas de transición son probabilísticas (azar controlado por los parámetros P_c y P_m), no determinísticas.

Ventajas

- "Caja negra": Son independientes del problema, no necesitan conocimientos expertos previos de las matemáticas subyacentes.
- Paralelismo: Al operar sobre múltiples soluciones a la vez, son muy fáciles de paralelizar en procesadores modernos.
- Robustez: Escapan fácilmente de los mínimos locales en superficies complejas.

Inconvenientes y cuando NO usar un AG

- Problema: Pueden sufrir convergencia prematura o tardar mucho tiempo en llegar a una solución.
- No usarlos si: Necesitas matemáticamente garantizar al 100% que has llegado al óptimo global.
- No usarlos si: Conoces la función y esta es suave, convexa y de una sola variable (los métodos analíticos como la derivada tardarán milisegundos, el AG tardaría mucho más para lo mismo).
- No usarlos si: El espacio de búsqueda es muy pequeño o limitado (en ese caso, un algoritmo de fuerza bruta que pruebe enumerando todas las opciones será más seguro y rápido).

Programación genética

Propuesta por John Koza en 1992, la Programación Genética (PG) es la evolución natural de los Algoritmos Genéticos (AG). Mantienen el mismo principio darwiniano, pero su objetivo es mucho más ambicioso: no buscan el valor de un parámetro, sino que inducen programas y algoritmos completos para resolver un problema.

Mientras los AG usan codificación lineal (cadenas de bits o vectores), la codificación en PG se realiza en forma de ÁRBOL. Esto es así porque imita cómo los compiladores informáticos leen los programas.

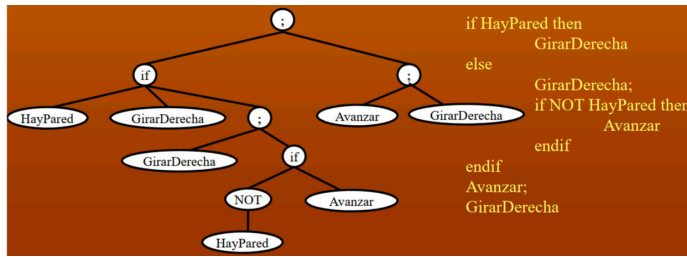
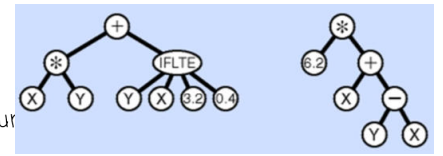
¿Qué puede representar un árbol en PG?

- Expresiones Aritméticas: Fórmulas matemáticas (ej: $2 \cdot \pi + (x+3)$).
- Expresiones Lógicas: Ecuaciones booleanas con AND, OR, NOT.
- Programas (Sistemas Basados en Reglas): El árbol representa un flujo lógico. La rama izquierda suele ser la condición (if) y la derecha la acción (then o else).

Operadores genéticos en árboles

Al cambiar la codificación, los operadores tienen que adaptarse a la estructura de árbol:

- **Cruce:** Se selecciona un nodo al azar en el "Padre 1" y otro en el "Padre 2". Lo que se hace es intercambiar los sub-árboles completos que cuelgan de esos nodos.
- **Mutación:** Se elige un nodo al azar de un individuo, se arranca ese sub-árbol y se sustituye por un nuevo generado aleatoriamente, o bien se insertan/duplican nodos.



Ejemplo

El Laberinto: Se busca un algoritmo que saque a un robot virtual de un laberinto.

- **Operadores (Nodos internos):** Secuencia (.), condición (if-then-else), negación (not).
- **Terminales (Hojas del árbol):** Acciones base como Avanzar, GirarDerecha, GirarIzquierda, HayPared.

Diseño de RNA

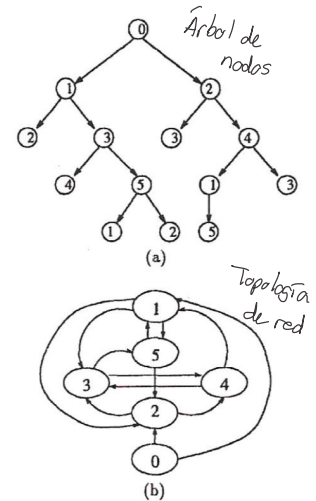
La Computación Evolutiva se aplica al diseño de RNA de tres maneras fundamentales:

- **Entrenamiento:** Búsqueda del conjunto óptimo de pesos de conexión.
- **Topología:** Búsqueda del número de neuronas y la conectividad óptima.
- **Construcción:** Uso de la Programación Genética (PG) para obtener reglas de construcción de redes.

Búsqueda de Topología y construcción mediante PG

A diferencia de los métodos de codificación directa (donde cada gen es una conexión específica), el uso de PG permite una codificación indirecta.

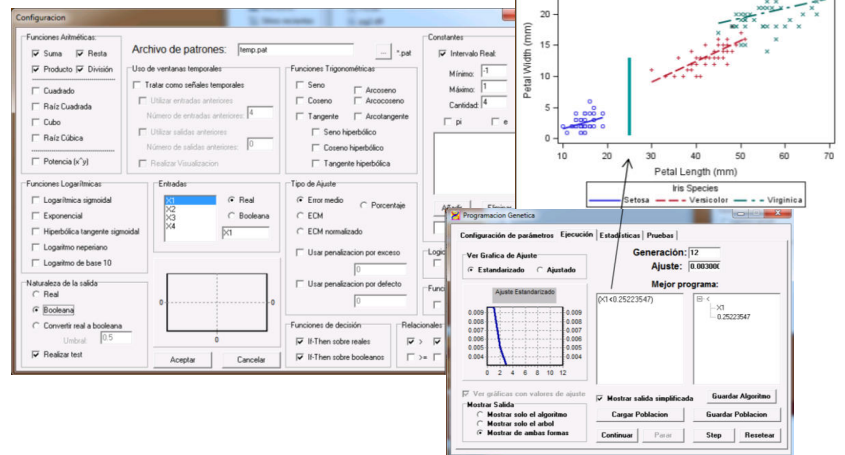
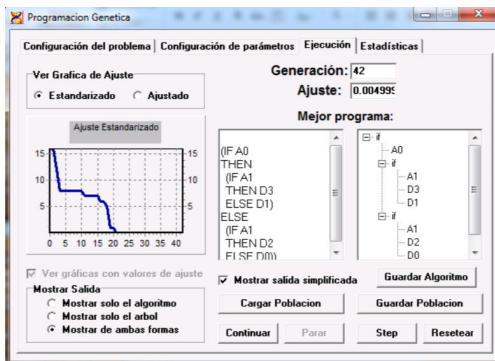
- **Representación:** La red se codifica como un árbol de nodos y conexiones.
- **Función de Evaluación (Fitness):** Se busca minimizar el error de la red, pero penalizando o ponderando el número de nodos para evitar estructuras excesivamente complejas e ineficientes.
- **Operadores Genéticos en Árboles:**
 - **Cruce:** Intercambio de subárboles entre dos redes progenitoras.
 - **Mutación:** Puede implicar el cambio de un nodo, la inserción de nuevos elementos o la duplicación de ramas existentes.



Ejemplos

Multiplexor (4 a 1): Evolución de un programa que decide qué entrada enviar a la salida basándose en señales de control.

Regresión Simbólica: Proceso de obtener reglas y ecuaciones matemáticas (patrones) a partir de un conjunto de datos brutos.



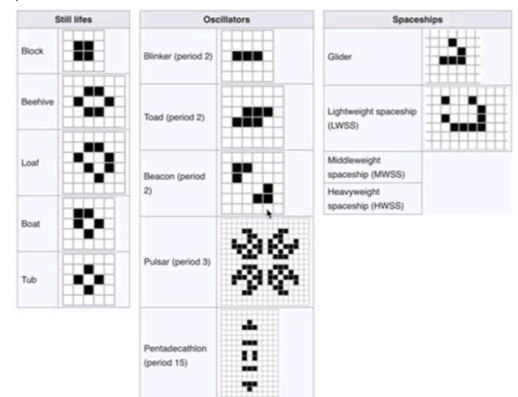
Vida artificial (VA)

La Vida Artificial es el estudio de sistemas creados por el hombre que imitan las propiedades de la vida biológica.

- **Diferencia clave con la CE:** Mientras la Computación Evolutiva busca resolver tareas específicas, la VA se centra en el estudio de sistemas complejos y procesos evolutivos en sí mismos.
- **Concepto de Emergencia:** Los comportamientos complejos surgen de la aplicación de reglas simples sin un control centralizado.

Elementos y modelos clásicos

1. **Auto-organización:** Sistemas que generan patrones de forma autónoma (ej. bandadas de pájaros).
2. **Boids (Craig Reynolds):** Simulación de bandadas basada en 3 reglas: Separación (evitar choques), Alineamiento (ir en la dirección media) y Cohesión (moverse hacia el centro del grupo).
3. **Juego de la Vida (Conway):** Autómata celular 2D donde cada estado depende del anterior según reglas estrictas:
 - **Nacimiento:** Una célula muerta revive si tiene exactamente 3 vecinas vivas.
 - **Supervivencia:** Una célula viva sigue viva si tiene 2 o 3 vecinas vivas.
 - **Muerte:** Por soledad (menos de 2 vecinas) o sobrepoblación (más de 3 vecinas).



Computación cuántica

Este nuevo paradigma surge por los límites físicos de la Ley de Moore. Al reducir los transistores a unos 7 nanómetros, los electrones dejan de ser predecibles bajo la física clásica y entran en el reino de la Mecánica Cuántica.

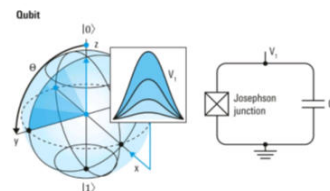
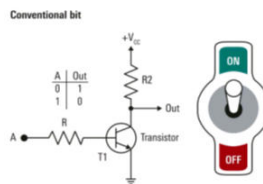
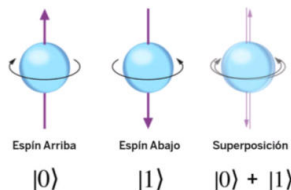
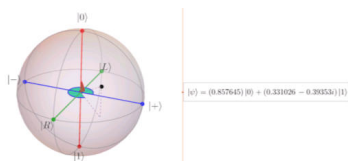
Principios fundamentales

- Dualidad: Los elementos subatómicos (cuantos) se comportan como onda y como partícula.
- Principio de Incertidumbre (Heisenberg): No se puede conocer con precisión absoluta la posición y el giro de una partícula a la vez; la propia medición interfiere en el resultado.
- Entrelazamiento: Dos partículas forman un único sistema; medir una determina instantáneamente el estado de la otra, sin importar la distancia.
- Superposición: Una partícula puede estar en varios estados a la vez. Solo "colapsa" en un estado concreto al realizar la medición.

El cúbit (Quantum bit)

A diferencia del bit convencional (0 o 1), el cúbit utiliza el espín del electrón para representar información.

- Estado: Representa una combinación lineal de los estados 0 y 1.
- Supremacía Cuántica: Debido a la superposición, 10 bits clásicos al $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$ 4 posibles; 10 cúbits almacenan los 1024 valores simultáneamente, permitiendo un paralelismo masivo.



A bit is a unit for measuring information		
Classical bits		Quantum bits (Qubits)
Bit 1 Empty = "0"	Bit 2 Filled = "1"	Qubit 1 1/2 of "0" and 1/2 of "1"
20 red beads = "0"	20 blue beads = "1"	
Head = "0"	Tail = "1"	
		50% chance of landing on "0" 50% chance of landing on "1"

Problemáticas y aplicaciones

Desafíos: Los cúbits son extremadamente frágiles (pierden el entrelazamiento fácilmente) y requieren temperaturas cercanas al cero absoluto (-273°C).

Aplicaciones:

- IA y Modelización: Sistemas complejos como el clima o biología.
- Criptografía Cuántica: La computación cuántica podría romper la criptografía clásica, pero también permite crear comunicaciones 100% seguras basadas en la superposición.